

---

## **Leveraging Edge AI for AI Decarbonization & Accessibility**

---

Mokorah Decra Kerubo

27/08/2024

## Contents

|                                                                                                          |          |
|----------------------------------------------------------------------------------------------------------|----------|
| Background                                                                                               | 3        |
| Problem Statement                                                                                        | 4        |
| Solution Statement                                                                                       | 4        |
| Scenario: Smart Agriculture System                                                                       | 5        |
| 1. Cloud Processing                                                                                      | 5        |
| Energy Consumption Breakdown:                                                                            | 5        |
| Total Energy Consumption:                                                                                | 6        |
| Carbon Footprint:                                                                                        | 6        |
| 2. Edge Processing                                                                                       | 6        |
| Energy Consumption Breakdown:                                                                            | 6        |
| Total Energy Consumption:                                                                                | 6        |
| Carbon Footprint:                                                                                        | 6        |
| Summary of Differences                                                                                   | 6        |
| Conclusion                                                                                               | 7        |
| System Functions/ Process Flow                                                                           | 7        |
| Practical Example: Deploying an AI Model for Crop Health Monitoring on a Raspberry Pi Using the Platform | 7        |
| Step 1: Data Collection                                                                                  | 8        |
| Step 2: Data Preprocessing                                                                               | 8        |
| Step 3: Model Training                                                                                   | 8        |
| Step 4: Model Optimization                                                                               | 8        |
| Step 5: Model Deployment                                                                                 | 8        |
| Step 6: Real-Time Monitoring and Updates                                                                 | 9        |
| Benefits of this Approach:                                                                               | 9        |
| Similar System:                                                                                          | 9        |
| Current Considerations                                                                                   | <b>9</b> |

# Background

Artificial intelligence (AI) involves the replication of human intelligence in technological systems. Prior to the concept of AI being fully understood and the term coined by John McCarthy, computers existed but were limited to following human commands. For example, if someone wanted to harness a fluctuating wind, they would manually turn a turbine to the south after observing that the wind from the north was stronger at that moment. With AI, however, the turbine is programmed to understand its goal that includes facing away from the strongest wind and it automatically turns south to catch the northern gust, eliminating the need for human intervention. The automation ideation included lots of popular names and many ideas such as the Imitation Game by Alan Turing to measure computers' intelligence up to the first Checkers game by Arthur Samuel in 1952 where the system mastered and could play the game independently<sup>1</sup>. Fast forward, AI has evolved in three major shapes, from simple systems, to transformers and the current Large Language models such as GPT3. This shift has equivalently recorded a proportionate increase in the energy consumption throughout the entire AI pre-development, usage and operational journey.

Energy consumption in AI is experienced in different stages that include;

- a. Developmental phase; wherein the model's development process, resources and conditions consume different amounts of energy. Energy use is driven by factors such as model size or number of parameters, computational resources, training hours, infrastructure, hyperparameter tuning, computational algorithms, data processing, training duration, and training energy<sup>2</sup>.
- b. Inference phase; wherein energy is consumed proportionate to usage and is primarily related to inference energy and evaluation energy. During this stage, the trained model is deployed and used for real-time tasks, where energy is consumed during inference.

*Some Statistic: Consumption of energy per AI prompt equals to a Full Smartphone worth of battery charge which is equivalent to leaving your refrigerator on for 30 minutes<sup>3</sup>.*

---

<sup>1</sup> "What Is the History of Artificial Intelligence (AI)?," Tableau, n.d., <https://www.tableau.com/data-insights/ai/history#:~:text=Birth%20of%20AI%3A%201950%2D1956&text=into%20popular%20use->

<sup>2</sup> Sourabh Mehta, "How Much Energy Do LLMs Consume? Unveiling the Power behind AI," Association of Data Scientists, July 3, 2024, <https://adasci.org/how-much-energy-do-llms-consume-unveiling-the-power-behind-ai/>.

<sup>3</sup>

<https://www.technologyreview.com/2023/12/01/1084189/making-an-image-with-generative-ai-uses-as-much-energy-as-charging-your-phone/>

## Problem Statement

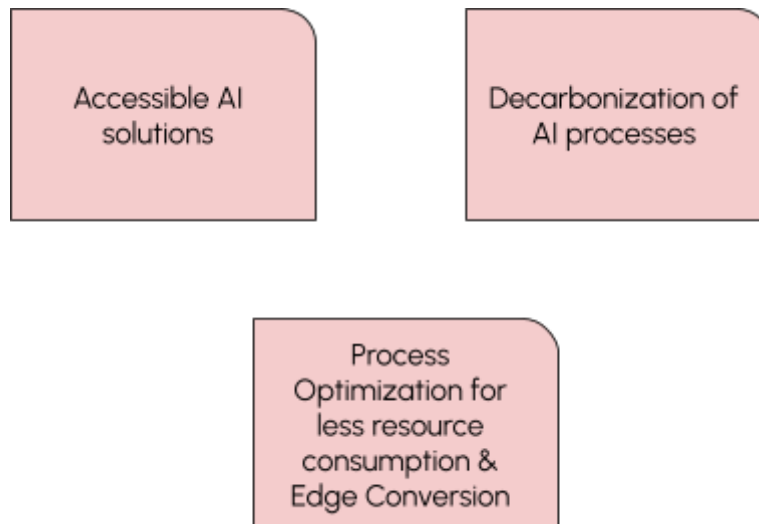
As artificial intelligence (AI) continues to advance and integrate into various sectors, the resources required for building and using AI models have significantly increased. This growing demand for computational power leads to substantial energy consumption, predominantly sourced from electricity grids that may rely on fossil fuels. Consequently, the energy footprint of AI systems continues to increase, ripping off undersupplied regions and even contributing to a carbon footprint surge.

Despite the promising applications of AI in optimizing various aspects of society, including energy management and climate modeling, there is inadequate adoption of alternative energy efficient development techniques such as edge computing.

In addition to this, most of these solutions, while effective, lack the development that leads them to be effectively used and accessed in low tech supply targets leaving underserved communities for whom these solutions are developed un-intervened for.

## Solution Statement

The solution focuses on creating a reliable platform for optimizing artificial intelligence (AI) models for edge devices, enhancing real-time monitoring and analysis capabilities. By utilizing edge computing, the platform minimizes energy consumption and reduces carbon emissions associated with traditional cloud-based AI systems. This approach addresses the growing need for accessible and energy-efficient AI solutions, particularly in underserved regions with limited internet connectivity, allowing users to leverage advanced technology without the high energy demands of centralized data processing.



### Practical example of the potential of Edge (Generated by AI - Can justify with more research)

Practical example involving a smart agriculture system that monitors soil moisture levels to optimize irrigation. We'll calculate and break down the energy usage and carbon emissions for both processing methods.

#### **Scenario: Smart Agriculture System**

##### **1. Assumptions:**

- **Data Collected:** A sensor collects soil moisture data every minute.
- **Data Size:** Each data point is approximately 1 KB.
- **Number of Sensors:** 100 sensors deployed across a farm.
- **Processing Frequency:** Data is processed every minute.
- **Energy Consumption:**
  - **Cloud Data Center:** 400 watts per server; cooling and operational overhead require 2x the server energy.
  - **Edge Device:** 5 watts per device for processing.
- **Data Transmission:**
  - **Cloud:** 0.1 kWh per GB transferred.
  - **Edge:** Minimal transmission needed, assume 0.01 kWh for periodic updates.

##### **1. Cloud Processing**

#### **Energy Consumption Breakdown:**

- **Data Transmission:**
  - Data collected per minute from 100 sensors = 100 KB/min.
  - Data collected in one hour = 100 KB \* 60 min = 6,000 KB = 6 MB/hour.
  - **Annual Data:** 6 MB \* 24 hours \* 365 days ≈ 52.56 GB.
  - Energy for data transmission: 52.56 GB \* 0.1 kWh/GB = 5.256 kWh.

- **Data Center Operation:**
  - **Number of Servers:** Assume 1 server can handle data from 500 sensors, so we need 1 server.
  - **Energy for Server Operation:**
    - Total server energy for a year (including cooling) = 400 watts \* 2 (for cooling) \* 24 hours/day \* 365 days = 7,008 kWh.

#### Total Energy Consumption:

- Total energy = Data transmission + Data center operation = 5.256 kWh + 7,008 kWh = **7,013.256 kWh/year.**

#### Carbon Footprint:

- Assuming the carbon intensity of electricity is 0.5 kg CO<sub>2</sub>/kWh.
- Total CO<sub>2</sub> emissions = 7,013.256 kWh \* 0.5 kg CO<sub>2</sub>/kWh = **3,506.628 kg CO<sub>2</sub>/year.**

## 2. Edge Processing

#### Energy Consumption Breakdown:

- **Data Transmission:** Minimal, assume only periodic summaries sent to the cloud for analytics.
  - Annual data transmission energy = 0.01 kWh (estimated minimal updates) = 0.01 kWh/year.
- **Edge Device Operation:**
  - **Energy for 100 devices:** 5 watts/device.
  - Total energy for devices = 5 watts \* 100 devices \* 24 hours/day \* 365 days = 438 kWh/year.

#### Total Energy Consumption:

- Total energy = Data transmission + Edge device operation = 0.01 kWh + 438 kWh = **438.01 kWh/year.**

#### Carbon Footprint:

- Total CO<sub>2</sub> emissions = 438.01 kWh \* 0.5 kg CO<sub>2</sub>/kWh = **219.005 kg CO<sub>2</sub>/year.**

#### Summary of Differences

| Factor              | Cloud Processing             | Edge Processing    |
|---------------------|------------------------------|--------------------|
| Data Transmission   | High (5.256 kWh)             | Minimal (0.01 kWh) |
| Computational Power | High (7,008 kWh)             | Lower (438 kWh)    |
| Cooling Systems     | Significant (included above) | None needed        |

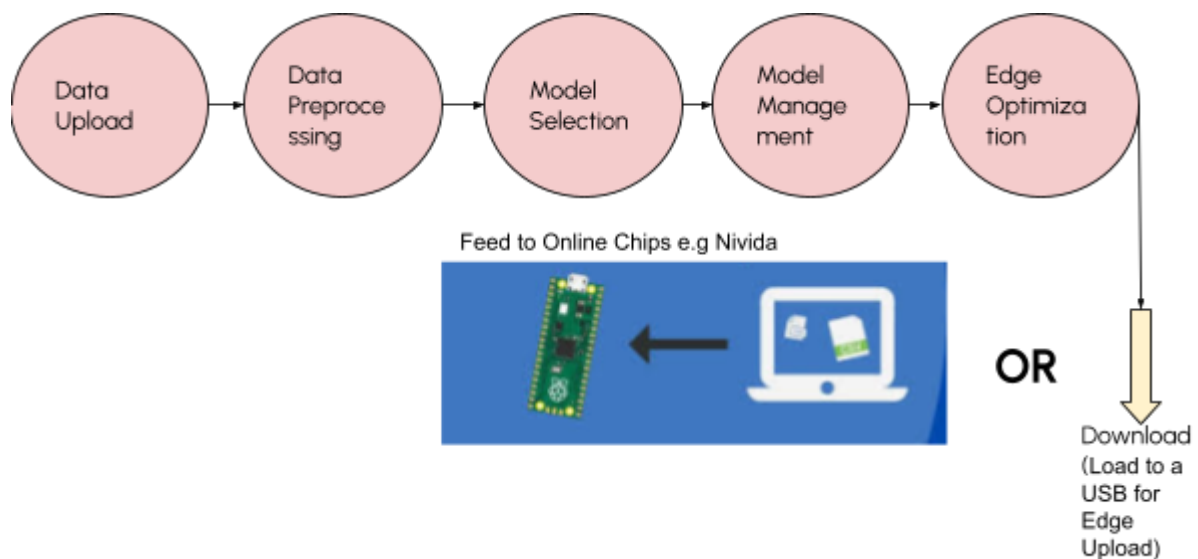
|                                           |                                   |                            |
|-------------------------------------------|-----------------------------------|----------------------------|
| <b>Total Energy Consumption</b>           | <b>7,013.256 kWh/year</b>         | <b>438.01 kWh/year</b>     |
| <b>Carbon Footprint</b>                   | <b>3,506.628 kg CO2/year</b>      | <b>219.005 kg CO2/year</b> |
| <b>Accessibility in Underserved Areas</b> | Limited by network infrastructure | Highly accessible          |

## Conclusion

Edge processing leads to significantly lower energy consumption and a smaller carbon footprint due to:

1. **Reduced Data Transmission:** Minimal data is sent to the cloud, saving energy in network infrastructure.
2. **Energy-Efficient Devices:** Edge devices consume significantly less power compared to large data centers.
3. **Elimination of Data Center Energy Use:** Edge processing negates the need for energy-intensive cooling and operational power in data centers.

## System Functions/ Process Flow



## Use Case Scenario

Practical Example: Deploying an AI Model for Crop Health Monitoring on a Raspberry Pi Using the Platform

## Step 1: Data Collection

You want to monitor the health of crops in real-time using a Raspberry Pi with a camera sensor. To do this, you would start by collecting data (images of healthy and unhealthy crops) using the camera connected to the Raspberry Pi.

- The camera on the Raspberry Pi captures images of crops.
- These images are uploaded to the platform through the Command Line Interface (CLI) or directly using the platform's interface if the Raspberry Pi is connected.

## Step 2: Data Preprocessing

Once the data (images) are collected, the platform allows you to preprocess it:

- For images, this may involve resizing, normalizing, or applying filters to enhance features.
- You might label the images as "healthy" or "unhealthy" depending on their condition.

## Step 3: Model Training

Using the platform's studio, you would:

- Select a machine learning model type, for example, a convolutional neural network (CNN) for image classification.
- Train the model using the labeled images. The platform automatically configures the model to work efficiently on edge devices.
- The model is trained and optimized to detect healthy vs. unhealthy crops from images.

## Step 4: Model Optimization

Once the model is trained, the platform optimizes the model for the Raspberry Pi (or any other edge device). This optimization is key because edge devices usually have limited processing power and memory. The platform ensures the model is small enough to fit within the device's constraints.

- For instance, it might convert the model into a TensorFlow Lite version, which is lightweight and efficient for the Raspberry Pi.

## Step 5: Model Deployment

After the model is trained and optimized, it's ready for deployment. Here's how the deployment works practically:

1. Download the model: The trained model is downloaded from the platform in a format compatible with the Raspberry Pi (e.g., TensorFlow Lite, or C++ SDK if needed).
2. Deploy to the Raspberry Pi:
  - You copy the model files to the Raspberry Pi via SSH, USB, or directly through the internet.



- Using the platform SDK or generated code, you load the model onto the Raspberry Pi.
- 3. Running Inference:
  - The Raspberry Pi now runs the model in real-time, using the camera feed to analyze images of crops.
  - The model processes the images locally on the Raspberry Pi, and outputs whether a crop is healthy or unhealthy without needing an internet connection (this is the on-device inference).

#### Step 6: Real-Time Monitoring and Updates

- The model makes predictions based on live camera feeds. For example, as it processes the camera data, it might notify a farmer of an unhealthy crop based on visual symptoms.
- If you collect new data, you can easily upload it to the platform for further training and refinement, and then re-deploy the updated model to the Raspberry Pi.

## Benefits of this Approach:

- Real-time processing: No need to send data to the cloud for processing. Everything happens on the edge device.
- Energy efficiency: Since the model is optimized for low-power devices, it consumes less energy.
- Scalability: Once the model is working well, it can be easily deployed to other edge hardware in different fields or locations.
- Accessibility: This system can work in areas with limited or no internet access because the model runs locally.
- Scalability: Data inference carbon footprint reduced to local processing.

## Similar System:

[Edge Impulse](#)

## Current Considerations

1. Work with only Image data for now.
2. Work with only agricultural pretrained images for now.
3. Supply edge hardware in my country alongside this.

## PRE- DEVELOPMENT PHASE

Primary Function - Optimize models for edge deployments

Requirements for successful edge deployments

| ELEMENT            | Typical | Required | Technique          | Tools to use                       | Other ways      |
|--------------------|---------|----------|--------------------|------------------------------------|-----------------|
| Memory             |         |          |                    |                                    |                 |
| Processing Power   |         |          |                    |                                    |                 |
| Energy             |         |          |                    |                                    |                 |
| Model Weight       | 32      | 8-16     | Model Quantization | TensorFlow Lite, ONNX, and PyTorch | Prune & Distill |
| Lightweight Models |         |          |                    |                                    |                 |

Data Collection

Data Annotation

Data Preprocessing

Model Selection

Feature Engineering

Model Training

Model Testing

Model Deployment/download